Software Engineering Lab

Lab 17

Architecture Decision Record & Validasi Struktural

Mencegah Perdebatan Berulang dan Kerusakan Silsilah Keputusan di Go



Lukman / lukman-ss

Konteks Keputusan Hilang Memicu Debat Teknis Berulang

Ketika alasan pemilihan desain masa lalu lenyap, tim terjebak dalam siklus refactoring kontraproduktif.

DAMPAK HILANGNYA REKAMAN ARSITEKTUR

Pembalikan Buta

Keputusan masa lalu dibalik tanpa mengerti batasan awal yang memicunya saat peluncuran.

DAMPAK HILANGNYA REKAMAN ARSITEKTUR

Dogma Usang

Pola lama dipertahankan secara buta meski beban traffic dan skala tim telah melompat 5× lipat.

DAMPAK HILANGNYA REKAMAN ARSITEKTUR

Bukan Berarti Malas

Tim aktif berdiskusi teknis, tetapi bukan berarti produktif jika memperdebatkan opsi yang sama berulang kali.

Ketika konteks historis arsitektur hilang, pengembang baru tidak mengetahui batasan teknis di balik pilihan desain awal. Diskusi intensif bukan berarti tim produktif jika mereka hanya mengulang perdebatan yang sama tanpa acuan rekam jejak formal.

Undocumented decisions trigger cyclical technical debates and blind reversal of proven architectural trade-offs.

Wiki dan Markdown Manual Cepat Mengalami Drift

Mencatat keputusan di luar repositori atau tanpa linter struktural menciptakan dokumen fiktif.

PENDEKATAN NAIF VS REALITA TEKNIS

Wiki Eksternal Terisolasi

Confluence dan Google Docs terpisah dari Git commit; segera usang saat kode di-refactor.

PENDEKATAN NAIF VS REALITA TEKNIS

Dangling References

ADR lama ditandai Superseded by 99, tetapi berkas 99 tidak pernah ditulis atau terhapus.

PENDEKATAN NAIF VS REALITA TEKNIS

Tidak Otomatis Akurat

Menulis format markdown di repo tidak otomatis valid tanpa aturan linter CI yang memverifikasi integritas graf.

Penyimpanan keputusan di wiki eksternal menyebabkan keterpisahan parah antara implementasi kode dan dokumentasi. Menaruh file markdown di repositori tidak otomatis menjaga kebenaran data jika tautan nomor dan siklus hidupnya tidak diuji mesin.

Unverified markdown ADRs accumulate broken links, phantom replacements, and invalid states over time.

Empat Pilar Siklus Hidup Architecture Decision Record

Keputusan berevolusi melalui penomoran monotonik dan riwayat penggantian timbal-balik.

VERSION CONTROLLED

1. Ko-lokasi Git

ADR disimpan bersama kode; berevolusi melalui mekanisme Pull Request dan Code Review.

IMMUTABLE HISTORY

2. Kekekalan Dokumen

ADR Accepted dilarang diedit retroaktif; arah baru wajib dicatat pada ADR baru.

DIRECTED GRAPH

3. Silsilah Berarah

Relasi timbal-balik: status Superseded by N wajib dibalas deklarasi Supersedes M.

LINTER INVARIANT

4. Validasi CI Otomatis

Hanya pada jalur ini validasi terotomatisasi mencegah nomor lompat dan siklus sirkular.

ADR mengabadikan keputusan penting ke dalam repositori dengan sifat kekal setelah disetujui. Hanya pada jalur ini, yakni lewat validasi pipeline otomatis, silsilah keputusan terhindar dari pembaruan sepihak dan inkonsistensi nomor.

Treat architectural decisions as immutable nodes in an append-only directed acyclic graph.

Pipeline Validasi: Dari Raw Markdown Menjadi Error List

Alur pemrosesan linear dari pembacaan teks, pemindaian regex, hingga pengujian paralel.

READER

1. Text Ingestion

Membaca string berkas markdown ADR dari repositori atau buffer memori.

ADR.PARSE

2. Regex Parsing

Mengekstrak ID, Title, Status, SupersededBy, Supersedes, dan seksi wajib.

ADR.LINTER

3. Monotonic Check

Memastikan kumpulan ID terurut 1..N tanpa duplikasi ID dan tanpa nomor lompat.

GOROUTINES

4. Graph & Parallel

Verifikasi referensi timbal-balik; belum membuktikan integrasi git hook otomatis.

Alur validasi memindai string dokumen, memverifikasi kelengkapan seksi teks, mengurutkan ID penomoran, lalu memverifikasi tautan graf. Pipeline ini membuktikan validasi in-memory, tetapi belum membuktikan integrasi langsung dengan git pre-commit hook.

Sequential structural parsing followed by concurrent referential graph integrity verification.

Ekstraksi Metadata dan Seksi Wajib Lewat Regex

Parser memastikan judul bernomor, status valid, dan tiga seksi wajib terisi.

```
var (
    titleRegex  = regexp.MustCompile(`^#\s+(\d+)\.\s+(.+)$`)
    statusRegex = regexp.MustCompile(`(?i)^Status:\s*([A-Za-z]+)(?:\s+by\s+(\d+))?`)
    contextRegex = regexp.MustCompile(`(?i)^##\s+Context`)
    decisionRegex = regexp.MustCompile(`(?i)^##\s+Decision`)
)
// Parse memverifikasi isi seksi Context, Decision, Consequences
if !foundContext || !contextHasContent { return nil, errors.New("context empty") }
```

Parser mengekstrak nomor ID dan memeriksa keberadaan konten pada seksi Context, Decision, dan Consequences. Menggunakan regex kaku bukan berarti parser fleksibel terhadap semua dialek Markdown; parser ini sengaja membatasi diri pada konvensi baku Nygard.

Strict regex validation enforces mandatory architectural sections before records enter the graph.

internal/adr/parser.go:11–36

Anatomi Kerusakan Graf: Tautan Sepihak & Referensi Hantu

Empat mode kegagalan silsilah keputusan yang langsung digagalkan oleh linter.

SKENARIO KEGAGALAN STRUKTUR DOKUMENTASI

Supersession Sepihak

ADR 2 mengklaim Supersedes: 1, tetapi ADR 1 masih berstatus Accepted tanpa tanda penggantian.

SKENARIO KEGAGALAN STRUKTUR DOKUMENTASI

Referensi Hantu

ADR 1 berstatus Superseded by 99, padahal ADR 99 tidak pernah ada di dalam repositori.

SKENARIO KEGAGALAN STRUKTUR DOKUMENTASI

Penomoran Bolong

ID melompat dari 1 langsung ke 3; menimbulkan keraguan apakah ada dokumen yang hilang.

SKENARIO KEGAGALAN STRUKTUR DOKUMENTASI

Self-Supersession

ADR 1 menandai dirinya digantikan oleh ADR 1; hanya pada jalur ini siklus konyol dicegah linter.

Kerusakan dokumentasi paling berbahaya terjadi saat penggantian dilakukan sepihak atau menunjuk ke dokumen hantu. Hanya pada jalur ini linter mendeteksi ketidaksesuaian relasi dan menggagalkan merge sebelum informasi keliru menyebar ke tim.

Broken supersession links create split-brain architectural states where obsolete designs appear active.

tests/linter_test.go:36-114

Matriks Komparasi: Gejala Anomali vs Pencegahan Mesin

Membedah bagaimana aturan linter mengatasi kegagalan dokumentasi manual secara terstruktur.

Anomali Dokumentasi	Dampak di Tim Engineering	Aturan Verifikasi Linter
Tautan Sepihak	Status ganda: dua arsitektur bentrok	Reciprocal Check (rec.Supersedes == old.ID)
Dangling Pointer	Konteks pengganti hilang tanpa jejak	Exist Check (replacement, exists := recordMap[ID])
Siklus Sirkular	Loop penggantian tanpa ujung (1->2->1)	DFS Cycle Detection (3-state coloring)
Nomor Tidak Urut	Ambiguitas file hilang dalam Git history	Monotonic Numbering Check (ids[i] == i+1)
Format Informal	Status 'Draft' / seksi wajib hilang	Parser Guard (IsValid() & Section Check)

Tabel ini memetakan dampak buruk anomali dokumentasi terhadap tim dan mekanisme kode yang mengatasinya. Pemeriksaan timbal-balik dan pewarnaan graf 3 status memastikan tidak ada rantai keputusan yang mengalami benturan status atau siklus berputar.

Map human documentation oversights to strict algorithmic invariants enforceable in compilation and CI.

Evolusi Arsitektur SaaS ERP: Dari Monolith ke Service

Tiga keputusan arsitektural berurutan yang menggambarkan siklus hidup nyata sistem.

SEKUENS KEPUTUSAN ARSITEKTUR LAB

ADR 0001: Modular Monolith

Dipilih saat tim 5 orang demi kecepatan iterasi dan integritas transaksi relasional.

SEKUENS KEPUTUSAN ARSITEKTUR LAB

ADR 0002: Extract Notification

Menggantikan ADR 1 saat tim 25 orang dan lonjakan burst traffic membebani core ERP.

SEKUENS KEPUTUSAN ARSITEKTUR LAB

ADR 0003: Reject Event Sourcing

Ditolak formal; bukan berarti anti Event Sourcing, melainkan tabel audit SQL sudah memadai.

Rangkaian keputusan memperlihatkan transisi matang dari Modular Monolith menuju ekstraksi microservice ketika beban meningkat. Menolak Event Sourcing pada ADR 3 bukan berarti anti inovasi, melainkan keputusan sadar menghindari kompleksitas skema yang belum dibutuhkan.

Record both accepted architectural migrations and rejected proposals to prevent unneeded complexity.

Eksekusi Test Suite dan Demo cmd/demo Lolos Bersih

Output terminal membuktikan seluruh 13 sub-test dan eksekusi demo berjalan deterministik.

```
=== RUN   TestLinter_ValidSequence
--- PASS: TestLinter_ValidSequence (0.00s)
=== RUN   TestLinter_BrokenReferences (7 sub-tests)
--- PASS: TestLinter_BrokenReferences (0.00s)
=== RUN   TestLinter_ConcurrencyStress (100 records)
--- PASS: TestLinter_ConcurrencyStress (0.00s)
=== RUN   TestParse_Valid & TestParse_Invalid (9 sub-tests)
--- PASS: TestParse_Invalid (0.00s)
PASS: ok lab17/tests 1.469s (race detector: clean)
=== Architecture Decision Record (ADR) Lab ===
Successfully parsed 3 ADRs | Linter: Integrity check passed!
```

Verifikasi terminal membuktikan seluruh unit test parser dan linter lolos tanpa kebocoran data race pada stress test 100 record. Demo cmd/demo berhasil memvalidasi silsilah evolusi SaaS ERP dari Monolith ke Microservice.

Executable verification provides undeniable proof of structural parsing and referential integrity.

cmd/demo/main.go:66-103

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan batasan verifikasi in-memory terhadap skenario skala enterprise.

BATAS PEMBUKTIAN SISTEM

YANG DIBUKTIKAN

Parsing regex teks ADR, validasi nomor urut monotonik, dan deteksi siklus silsilah.

BATAS PEMBUKTIAN SISTEM

YANG TIDAK DIBUKTIKAN

Integrasi file system riil, git hook commit gating, dan pembuatan berkas CLI otomatis.

BATAS PEMBUKTIAN SISTEM

BATAS KLAIM EMPIRIS

Lab belum membuktikan klaim kuantitatif reduksi waktu onboarding pada tim 100+ engineer.

Lab ini memverifikasi kebenaran parsing in-memory dan integritas graf timbal-balik secara matematis. Pengujian ini belum membuktikan integrasi git hook otomatis maupun dampak kuantitatif terhadap kecepatan onboarding tim berskala ratusan engineer.

Strictly separate mathematically verified data structure invariants from unproven organizational claims.

content/01-content-brief.md:33-41

Trade-off Penerapan ADR dan Validasi Terotomasi

Disiplin pencatatan menambah overhead di PR demi kejelasan jangka panjang.

KOMPROMI REKAYASA PENCATATAN ADR

Overhead Pull Request

Setiap perubahan arsitektur mewajibkan berkas ADR baru dan update status dokumen lama.

KOMPROMI REKAYASA PENCATATAN ADR

Rigiditas Format Teks

Seksi Context, Decision, dan Consequences wajib diisi; menolak catatan singkat informal.

KOMPROMI REKAYASA PENCATATAN ADR

Tidak Otomatis Akurat

Linter hanya mengecek sintaks; tidak otomatis menjamin bahwa keputusan teknis yang diambil tepat.

Penerapan ADR menambah beban penyusunan PR karena tim harus menyepakati kata-kata keputusan dan konsekuensi. Memiliki linter tidak otomatis menjamin keputusan arsitektur benar; linter hanya menjaga integritas struktural teks, bukan kualitas logika bisnis.

ADRs trade short-term pull request authoring overhead for long-term clarity and preserved technical context.

Validasi Graf Konkuren Menggunakan Goroutines & Mutex

Pemeriksaan referensi dieksekusi paralel dengan agregasi galat terlindungi thread-safe.

```
var wg sync.WaitGroup
for _, r := range records {
    wg.Add(1)
    go func(rec *Record) {
        defer wg.Done()
        // Evaluasi referensi timbal balik rec.Supersedes & SupersededBy
        if len(localErrs) > 0 {
            mu.Lock()
            errs = append(errs, localErrs...)
            mu.Unlock()
        }
    }(r)
}
wg.Wait()
```

Validasi simpul graf dieksekusi secara konkuren menggunakan goroutine per record dan agregasi mutex. Simulasi ini tidak sama dengan operasi disk I/O riil pada ribuan berkas, namun membuktikan kebersihan dari race condition di memori.

Validate independent graph vertices concurrently while protecting error collectors behind mutex locks.

Transformasi: Dari Catatan Liar ke Graf Tervalidasi

Pergeseran dari dokumentasi pasif yang cepat usang menuju rekaman aktif di CI.

SEBELUM (MANUAL WIKI) VS SESUDAH (AUTOMATED ADR)

Sebelum: Wiki Liar

Keputusan tercecer di Slack atau Confluence; tautan rusak dan status usang tanpa peringatan.

SEBELUM (MANUAL WIKI) VS SESUDAH (AUTOMATED ADR)

Sesudah: Git ADR + Linter

ADR berdampingan dengan kode; CI memvalidasi keutuhan nomor dan tautan silsilah.

SEBELUM (MANUAL WIKI) VS SESUDAH (AUTOMATED ADR)

Bukan Berarti Kaku

Format terstruktur bukan berarti birokrasi berlebih; template minimal Nygard cukup 1 halaman.

Sebelum ada ADR, alasan pemilihan teknologi tercecer di arsip chat dan dokumen wiki yang tidak terawat. Menerapkan ADR dan linter bukan berarti menciptakan birokrasi kaku, melainkan mengunci kejelasan keputusan penting dengan usaha minimal.

Replace fragmented external documentation with version-controlled, lint-validated decision records.

Enforcing State Machine Siklus Hidup ADR

Mendefinisikan status resmi: Proposed, Accepted, Superseded, Deprecated, Rejected.

```
type Status string
const (
    StatusProposed   Status = "Proposed"
    StatusAccepted   Status = "Accepted"
    StatusSuperseded Status = "Superseded"
    StatusDeprecated Status = "Deprecated"
    StatusRejected   Status = "Rejected"
)
func (s Status) IsValid() bool { /* switch validasi */ }
```

Tipe data Status mengikat status ADR hanya pada lima kondisi siklus hidup formal. Keputusan membatasi status ini cakupannya mengikuti test spesifik lab, mencegah masuknya istilah rancu seperti In-Review atau Draft ke cabang utama.

Constrain document lifecycle states to a closed enumerated set validated at parse time.

Invarian Penomoran Monotonik Mencegah Berkas Hilang

Memastikan rangkaian ID terurut sempurna dari 1 sampai N tanpa jeda nomor.

```
ids := make([]int, 0, len(records))
for id := range recordMap { ids = append(ids, id) }
sort.Ints(ids)
for i := 0; i < len(ids); i++ {
    if ids[i] != i+1 {
        errs = append(errs, fmt.Errorf("non-monotonic numbering"))
        break
    }
}
```

Linter mengumpulkan seluruh ID dan mencocokkan nilainya tepat dengan indeks i+1 setelah diurutkan. Urutan penomoran yang rapi tidak otomatis menjamin tidak ada commit yang salah rebase, tetapi mencegah celah nomor yang membingungkan auditor.

Sequential monotonic numbering verifies that no historical decision records were deleted or omitted.

Deteksi Siklus Silsilah dengan Algoritma DFS 3-State

Mencegah loop penggantian melingkar yang merusak validitas historis arsitektur.

```
state := make(map[int]int) // 0: unvisited, 1: visiting, 2: visited
detectCycle = func(id int) {
    state[id] = 1
    if rec, ok := recordMap[id]; ok && rec.SupersededBy != 0 {
        nextID := rec.SupersededBy
        if state[nextID] == 1 {
            errs = append(errs, fmt.Errorf("cyclical supersession"))
        } else if state[nextID] == 0 { detectCycle(nextID) }
    }
    state[id] = 2
}
```

Deteksi siklus memanfaatkan algoritma pewarnaan simpul DFS tiga status: unvisited, visiting, dan visited. Hanya pada jalur ini deteksi rekursif mampu menangkap skenario loop kompleks di mana ADR 1 digantikan 2, 2 digantikan 3, dan 3 kembali menunjuk 1.

Detect invalid directed cycles using depth-first graph coloring before supersession chains corrupt lineage.

Verifikasi Timbal-Balik: Konsistensi Hubungan Penggantian

Memastikan ADR lama dan ADR baru saling mengakui hubungan silsilah penggantian.

```
if rec.Status == StatusSuperseded {
    replacement, exists := recordMap[rec.SupersededBy]
    if !exists { localErrs = append(localErrs, fmt.Errorf("non-existent")) }
    else if replacement.Supersedes != rec.ID {
        localErrs = append(localErrs, fmt.Errorf("mismatched supersession"))
    }
}
if rec.Supersedes != 0 && (old.Status != StatusSuperseded || old.SupersededBy != rec.ID) {
    localErrs = append(localErrs, fmt.Errorf("not marked as superseded"))
}
```

Linter memeriksa keselarasan dua arah antara SupersededBy dan Supersedes. Menjaga kedua deklarasi tetap sinkron bukan berarti menambah redudansi sia-sia, melainkan memastikan pembaca dokumen lama maupun baru mendapatkan visibilitas status yang utuh.

Reciprocal supersession verification guarantees that obsolete and current records remain mutually consistent.

Penolakan Berkas Cacat: Parser Guard Seksi Wajib

Memvalidasi bahwa seksi Context, Decision, dan Consequences bukan hanya header kosong.

```
func TestParse_Invalid(t *testing.T) {
    tests := []struct { name, content, wantErr string }{
        {"missing title", "Status: Accepted", "title not found"},
        {"invalid status", "# 1. T\nStatus: Draft", "invalid status: Draft"},
        {"empty context", "# 1. T\nStatus: Accepted\n## Context\n## Decision\nB...",
         "context section missing or empty"},
    }
    for _, tt := range tests { /* assert strings.Contains(err, wantErr) */ }
}
```

Pengujian unit memverifikasi bahwa parser menolak dokumen dengan judul cacat, status informal, atau seksi wajib tanpa konten. Aturan validasi ketat ini cakupannya mengikuti test suite yang menjamin setiap ADR memiliki bobot substansi arsitektur nyata.

Guard against hollow architectural documentation by validating mandatory content within each section.

tests/parser_test.go:104–219

Checklist Tata Kelola Architecture Decision Record

Standar wajib sebelum menggabungkan keputusan arsitektur baru ke repositori.

- ✓ ADR disimpan langsung di repositori Git dalam folder docs/adr/ berdampingan dengan kode.
- ✓ Penomoran berkas menggunakan format monotonik 4 digit tanpa jeda (0001, 0002, ...).
- ✓ Status keputusan strictly terbatas pada: Proposed, Accepted, Superseded, Deprecated, Rejected.
- ✓ Dokumen yang telah Accepted bersifat immutable; pembaruan arah wajib membuat berkas ADR baru.
- ✓ Penggantian keputusan mendeklarasikan tautan timbal-balik (Superseded by N dan Supersedes M).
- ✓ Linter otomatis dijalankan pada pipeline CI untuk memvalidasi nomor, status, dan integritas graf.

Enforce structured decision governance to maintain an immutable, verifiable audit trail of system design.

Anti-Pattern Umum dalam Dokumentasi Arsitektur

Kekeliruan perancangan yang sering merusak reliabilitas rekaman keputusan tim.

HINDARI KESALAHAN UMUM INI

Penyuntingan Retroaktif

Mengubah isi ADR lama saat arah berubah; menghancurkan riwayat konteks historis arsitektur.

HINDARI KESALAHAN UMUM INI

ADR untuk Hal Sepele

Mencatat refactor nama variabel atau library helper kecil; menciptakan kebisingan dokumen.

HINDARI KESALAHAN UMUM INI

Bukan Berarti Bebas

Menolak ADR formal bukan berarti bebas bergerak; tim hanya menunda biaya debat di masa depan.

Mengedit ADR lama yang sudah disetujui menghapus jejak sejarah dan memicu kebingungan mengapa keputusan itu dibuat awalnya. Menolak menulis ADR bukan berarti tim lebih gesit; hal itu hanya menunda biaya debat berkepanjangan ketika arsitek berganti.

Never edit historical decisions; append new records and update status links to reflect system evolution.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Verifikasi mandiri parser dan linter ADR tanpa ketergantungan library eksternal.

```
cd labs/17-architecture-decision-record
go test -v ./...    # validasi parser regex dan aturan linter struktural
go test -race ./... # verifikasi validasi graf konkuren bebas data race
go run ./cmd/demo   # eksekusi demo parsing 3 ADR evolusi SaaS ERP
```

Seluruh mekanisme validasi ditulis menggunakan Go standard library tanpa modul eksternal. Eksperimen lokal ini memvalidasi integritas data di memori, namun production masih perlu menambahkan skrip CLI berkas dan integrasi hook commit.

Zero-dependency algorithmic verification guarantees fast, reliable validation in local and CI workflows.

README.md:14-34

Prinsip Inti: Keputusan Adalah Kode yang Harus Diuji

Arsitektur bertahan lama ketika konteks, batasan, dan konsekuensi dirawat layaknya software.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Ko-lokasi dengan Kode

Simpan rekam jejak keputusan di cabang Git yang sama agar selalu berevolusi bersama fitur.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Kekekalan Silsilah

Perlakukan riwayat arsitektur sebagai append-only log; ubah arah hanya melalui dokumen pengganti.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Verifikasi Otomatis

Dokumentasi tanpa tes tidak otomatis dipercaya; uji keutuhan graf keputusan di pipeline CI.

Dokumentasi keputusan arsitektur harus diperlakukan sama dengan kode sumber aplikasi. Menyimpan dokumen tanpa mekanisme pengujian otomatis tidak otomatis membuatnya berguna; keandalan dokumentasi hadir ketika strukturnya divalidasi oleh compiler dan CI.

Treat architectural records as an append-only, machine-verified ledger of technical compromises.

Dokumentasi arsitektur yang tidak diuji mesin adalah dokumen yang perlahan berubah menjadi kebohongan.

Architectural documentation unverified by machines gradually drifts into fiction.

Software Engineering Lab 17 – Architecture Decision Record & Validasi Struktural

internal/adr/models.go • internal/adr/parser.go • internal/adr/linter.go
cmd/demo/main.go • tests/parser_test.go • tests/linter_test.go
engineering/01–design.md • content/02–master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/17-architecture-decision-record`

Lukman / lukman-ss

Commit: 36ff6c8 • Status: APPROVED
Test passing: 13 test scenarios (monotonic sequence, reciprocal links, DFS cycle detection)
Race detector: Clean (go test -race ./... PASS in 1.47s)
Zero external dependencies: 100% Go standard library.